

User Management API — Create Users & Set Attributes (Customer workflow)

Owner	G Guilherme Menezes
Tags	

Overview

This guide walks through the **customer workflow** for **creating users** and **managing user attributes** via the GraphQL API. User attributes are key-value pairs attached to a user that drive features like **parameterized connection resolution**, **row-level authorization**, and **personalized data access**.

Workflow in short:

1. Create one or more users with role assignments and (optionally) attributes.
2. Update attributes on existing users as needed.
3. Use **impersonateUser** to start a session for that user — attributes set on the user are automatically available, or pass additional transient attributes at impersonation time.

Prerequisites

- **GraphQL endpoint:** `{{BASE_URL}}/graphql` (e.g. `https://canary.gowisdom.ai/graphql` or your deployment URL).
- **Access token:** A valid bearer token with `iam:write` permission. Typically obtained via your auth system (e.g. Descope) or the `exchangeAccessToken` query.
- **Workspace ID** (optional): Required only for multi-workspace deployments. Omit for single-workspace setups.

Step 1: Create users

Use **createUsers** to invite one or more users by email. You can assign roles, set initial attributes, add them to user groups, and optionally send a welcome email — all in a single call.

Input schema

```
CreateUsersInput {
  emails: [String!]!           # One or more email addresses
  roleAssignments: [RoleAssignmentInput!]! # Roles to assign
  workspaceID: ID              # Optional – for multi-workspace
  sendWelcomeEmail: Boolean    # Optional – defaults to false
  userAttributes: [UserAttributeInput!]    # Optional – key/value pairs
  userGroupIDs: [ID!]          # Optional – add to existing groups
}

RoleAssignmentInput {
  roleId: ID!                  # The role ID to assign
  scopes: [String!]!           # Scope IDs (domain IDs, workspace IDs, etc.)
}

UserAttributeInput {
  key: String!
  value: String!
}
```

Example: create a user with attributes

```

curl -s "{{BASE_URL}}/graphql" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT" \
  -d '{
    "query": "mutation CreateUsers($input: CreateUsersInput!)
{ createUsers(input: $input) { status } }",
    "variables": {
      "input": {
        "emails": ["alice@example.com"],
        "roleAssignments": [
          {
            "roleId": "{{ROLE_ID}}",
            "scopes": ["{{DOMAIN_ID}}"]
          }
        ],
        "sendWelcomeEmail": true,
        "userAttributes": [
          { "key": "account_id", "value": "acct_12345" },
          { "key": "region", "value": "us-east" }
        ]
      }
    }
  }'

```

Example: create multiple users at once

Pass multiple emails in the same call. All users receive the same roles and attributes.

```

curl -s "{{BASE_URL}}/graphql" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT" \
  -d '{
    "query": "mutation CreateUsers($input: CreateUsersInput!)
{ createUsers(input: $input) { status } }",

```

```

    "variables": {
      "input": {
        "emails": ["alice@example.com", "bob@example.com", "carol@example.com"],
        "roleAssignments": [
          {
            "roleId": "{{ROLE_ID}}",
            "scopes": ["{{DOMAIN_ID}}"]
          }
        ],
        "userAttributes": [
          { "key": "account_id", "value": "acct_12345" }
        ]
      }
    }
  }
}'

```

Step 2: Update user attributes

Use **setUserAttributes** to update attributes on an existing user at any time. This replaces all **DATABASE-sourced attributes** for that user with the provided list.

Input schema

```

SetUserAttributesInput {
  userId: ID! # The Wisdom user ID
  userAttributes: [UserAttributeInput!]! # New set of attributes
}

```

Example: update attributes for a user

```
curl -s "{{BASE_URL}}/graphql" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT" \
  -d '{
    "query": "mutation SetUserAttributes($input: SetUserAttributesInput!) { setUserAttributes(input: $input) { status } }",
    "variables": {
      "input": {
        "userId": "{{USER_ID}}",
        "userAttributes": [
          { "key": "account_id", "value": "acct_12345" },
          { "key": "region", "value": "eu-west" },
          { "key": "property_ids", "value": "prop_1,prop_2,prop_3" }
        ]
      }
    }
  }'
```



Important: `setUserAttributes` replaces all DATABASE-sourced attributes. Always include the full desired set, not just the ones you want to change. Attributes sourced from JWT (SSO claims) are not affected.

Step 3: Query users and their attributes

To verify attributes or look up user IDs, query the **users** list.

```
curl -s "{{BASE_URL}}/graphql" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT" \
  -d '{
```

```

    "query": "query ListUsers($input: ListUsersInput) { users
(input: $input) { id email name userAttributes { key value so
urce } roleAssignments { role { id name } scopes { id name sc
opeType } } } }",
    "variables": {
      "input": {
        "emailSearch": "alice@example.com"
      }
    }
  }
}'

```

Each attribute includes a **source** field:

- **DATABASE** — set via `createUsers`, `createPasswordAuthUser`, or `setUserAttributes`.
- **JWT** — derived from the user's authentication token (SSO claims). These cannot be changed via the API.

Step 4: Delete users from a workspace

Use `deleteUsersFromWorkspace` to remove one or more users from a specific workspace. This requires the `iam:delete` permission scoped to the target workspace.

Parameter	Type	Required	Description
<code>ids</code>	<code>[ID!]!</code>	Yes	Array of Wisdom user IDs to delete
<code>workspaceID</code>	<code>ID!</code>	Yes	The workspace to remove users from

Example: delete users from a workspace

```

curl -s "{{BASE_URL}}/graphql" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT" \
  -d '{
    "query": "mutation DeleteUsers($ids: [ID!]!, $workspaceI
D: ID!) { deleteUsersFromWorkspace(ids: $ids, workspaceID: $w

```

```
workspaceID) { status } }",
  "variables": {
    "ids": ["{{USER_ID_1}}", "{{USER_ID_2}}"],
    "workspaceID": "{{WORKSPACE_ID}}"
  }
}'
```



Important: This action is irreversible. Deleted users lose all role assignments, attributes, and access to the workspace. Make sure you have the correct user IDs before calling this mutation.

How user attributes work

1. **Two sources:** Attributes come from either the **database** (set explicitly via API) or the **JWT** (SSO token claims). Both are merged and available at runtime.
2. **Parameterized connections:** When a user makes a query, their attributes are matched against connection **credentialMappings.attributeMatch** to resolve which alternate connection config to use. See the [Parameterized Connections guide](#) for details.
3. **Impersonation:** When calling **impersonateUser**, you can pass additional transient attributes. These are merged with the user's stored attributes for that session.
4. **Row-level authorization:** Attributes can be referenced in row-level authorization rules to filter data per user.



Attributes that are **stable for a given user** (such as `account_id`) should be set permanently on the user via `createUsers` or `setUserAttributes`. Attributes that **change per session** (such as a selected view or context) can be passed transiently via `impersonateUser`.

